루프 엔지니어링으로 반복적인 업무를 자동화하기

마크다운 스펙으로 테트리스를 발전시키며 배우는 반복 설계

안녕하세요,
만나뵙게되어 영광입니다!

- 저는 이은솔이라고 합니다.
- MBTI는 ENTP 입니다.
- 웹부터 임베디드까지 모두 해본 6년차 개발자입니다.
- Claude Code를 활용해 1인 사업체를 운영하고 있습니다.
- 인생의 절반, 커리어의 전부를 삼성 핸드폰과 함께 했습니다.

하루라는 짧고도 긴 시간동안 잘 부탁드립니다!



강의의 방향성

루프 엔지니어링과 친해지는 첫 걸음

- 오늘은 마크다운 스펙을 한 장씩 설계/작성하여 **테트리스를 점점 발전**시킵니다.
- 게임을 잘 만든다기보다는, 기능을 작게 정의, 실행, 검증, 개선하는 흐름을 연습하는것에 집중합니다.
- 강의가 끝나면 테트리스 코드뿐 아니라, **직접 만든 하네스와 루프의 결과물**을 가져갑니다.

강의 순서

개념과 준비

1. 프롬프트부터 루프까지, AI 엔지니어링의 변천사
2. Claude Code 설치와 하네스 환경설정
3. 좋은 스펙과 완료 조건 설계하기

실습과 적용

4. 검증 방법과 상태 기록 준비하기
5. SPEC 00~03으로 테트리스 발전시키기
6. 선택 실습 SPEC 04 > 05+
7. 결과 공유와 마무리
8. Q&A

01

루프 엔지니어링의 이해

프롬프트가 못 넘은 벽, 하네스가 못 넘은 벽

루프 엔지니어링은 자동화 프로그램이 아닙니다

우리의 업무를 작은 단위로 쪼갠 반복적인 흐름입니다.

- 한 번 실행하고 끝나는 것이 아니라, **반복하고, 확인하고, 멈추고, 기록하는 흐름**을 만듭니다.
- 어렵게 들리지만 우리가 매일 하는 일과 같습니다. 일하고, 결과를 보고, 고치고, 다시 하는 것이죠.
- 프롬프트, 하네스, 루프 엔지니어링은 유행이 아닌, 점점 발전해나가는 하나의 큰 흐름입니다.

1세대 · 프롬프트 엔지니어링

한 번의 요청을 선명하게 만드는 기술

프롬프트가 해결한 것

- 먼저 정한 성공 기준
- 목표와 필요한 맥락·입력
- 작업 범위와 금지 조건
- 원하는 출력 형식
- 필요한 경우 좋은 예시

말로는 강제되지 않는 것

- 파일과 도구에 실제로 접근하기
- 권한과 작업 경계를 지키게 하기
- 테스트가 실행됐는지 보장하기
- 다음 세션까지 상태를 보존하기
- 재실행과 중단을 스스로 결정하기

좋은 프롬프트는 길어서가 아니라, 여러 시도에서 같은 성공 기준을 만족해서 좋습니다.

그러나, **부탁은 규칙이 될 수 없습니다.**

2세대 · 하네스 엔지니어링

부탁을 규칙으로 바꾸는 실행 체계

하네스 구성	하는 일	오늘 실습의 예
지시·컨텍스트	필요한 규칙과 작업 기준 제공	`CLAUDE.md`, SPEC
도구·실행 환경	파일을 바꾸고 결과를 직접 관찰	파일, 브라우저, 테스트
권한·경계	허용 범위와 승인 지점 제한	프로젝트 폴더, 사람 승인
검증·피드백	결과를 실제 신호와 비교	`test.html`, 직접 플레이
상태·기억	실행 사이의 진행과 근거 보존	코드와 `MEMORY.md`

Claude Code가 기본 하네스를 제공하고, 우리는 **프로젝트 규칙·경계·검증 방식**을 그 위에 더합니다.

반복되는 부탁은 하네스의 장치로 바꿉니다

반복되는 실패	말로만 다시 부탁하기	하네스에 넣을 장치
프로젝트 규칙을 잊음	“이번에는 기억해줘”	프로젝트 지침, Skill
범위 밖 파일을 수정함	“이 폴더만 건드려”	작업 경계, 권한, 샌드박스
확인 없이 완료를 선언함	“꼭 테스트해줘”	필수 검사, 브라우저 검증
다음 세션이 진행을 모름	“이전 내용을 기억해”	상태 파일, Git
위험한 명령을 실행함	“조심해서 작업해”	승인 게이트, Hook

같은 실패가 반복되면 프롬프트만 길게 쓰지 않습니다. **다음에는 그 실패가 어려워지도록 실행 체계를 고칩니다.**

하네스도 넘지 못한 벽

한 번의 실행은 좋아졌지만, 실행과 실행 사이가 비어 있습니다

하네스가 답하지 못하는 질문	비어 있는 자리
지금 무엇부터 시작해야 하는가?	작업을 꺼내는 규칙
이번 결과를 보고 다음엔 무엇을 하는가?	결과를 다음 입력으로 바꾸는 규칙
언제까지 다시 시도하고, 언제 멈추는가?	반복 상한과 종료 조건
세션이 끊기면 무엇이 남는가?	대화 밖의 기억

하네스는 **한 번의 실행을 잘하게** 만듭니다. 여러 번의 실행을 잇는 일은 여전히 사람이 손으로 해야합니다.

이 이름들은 누가 붙였을까요

2026년 상반기, 넉 달 사이에 나온 네 개의 정의

하네스 — Agent = Model + Harness

"If you're not the model, you're the harness."

Vivek Trivedy · LangChain · 2026-03

하네스의 구성 — 가이드와 센서

"Guides (feedforward) steer it before it acts. Sensors (feedback) observe after."

Birgitta Böckeler · Thoughtworks · 2026-04

루프 엔지니어링

"Loop engineering is replacing yourself as the person who prompts the agent."

Addy Osmani · Google · 2026-06

내 일은 루프를 쓰는 것

"I don't prompt Claude anymore. I have loops that are running. My job is to write loops."

Boris Cherny · Anthropic, Claude Code · 2026-06

Trivedy, The Anatomy of an Agent Harness · Böckeler, Harness engineering for coding agent users · Osmani, Loop Engineering · Cherny, Why Coding Is Solved, and What Comes Next

3세대 · 루프 엔지니어링

다음 실행을 결정하는 시스템

작업을 고르고 > 하네스에 맡기고 > 독립적으로 검증하고 > 상태를 기록한 뒤 > 다음 작업이나 중단을 결정합니다.

사람이 매 턴을 붙잡을 때

- 사람이 할 일을 고릅니다.
- 사람이 다음 프롬프트를 씁니다.
- 사람이 결과를 읽고 다시 지시합니다.
- 진행 상태는 대화 안에 머뭅니다.

루프를 설계했을 때

- 시작 조건이 작업을 꺼냅니다.
- 하네스가 정해진 경계에서 실행합니다.
- 검증 결과가 다음 행동을 바꿉니다.
- 외부 기억이 실행 사이를 잇습니다.

사람은 사라지지 않습니다. **완료 조건·안전 경계·최종 판단을 설계하는 역할**로 이동합니다.

두 종류의 루프를 구분합니다

Claude Code 내부 루프

```
관찰 → 행동 → 관찰 → 반복
```

한 번의 요청 안에서 모델이 도구를 쓰며 스스로 도는 **에이전트 실행 루프**입니다. 우리가 설계하지 않아도 이미 돌고 있습니다.

우리가 설계하는 외부 루프

```
실행 → 검증 → 기록 → 다음 결정
```

여러 실행과 세션을 잇는 **업무 루프**입니다. 이건 아무도 대신 만들어주지 않습니다.

Claude Code를 오래 실행하는 것만으로는 부족합니다. 바깥의 **목표·검증·기억·종료 조건**까지 설계해야 합니다.

진짜 루프는 결과가 다음 행동을 바꿉니다



- 통과하면 다음 작업 또는 성공 상태로 이동합니다.
- 실패했고 반복 예산이 남았다면 원인이 있는 곳을 보완합니다.
- 진전이 없거나 새로운 판단이 필요하면 사람에게 넘깁니다.
- 최대 반복 횟수나 비용에 도달하면 중단합니다. 상한은 **검증할 수 있는 만큼**만 줍니다 — 자율성을 최대로 주는 것이 아니라, 멈추고 검사할 수 있을 만큼만 주는 것이 back pressure입니다.

같은 일을 다시 실행하는 것은 재실행에 불과합니다. 검증 결과가 다음 행동과 종료를 바꿀 때 업무 루프가 됩니다.

back pressure — Osmani, Own the Outer Loop (2026-07): "We want to grant them just enough autonomy that we have enough back pressure to stop them, regulate them, check their work."

왜 대화가 길어지면 무너질까요

기억은 대화 안에 있고, 대화에는 크기 제한이 있습니다

```
컨텍스트 윈도우가 차오름
  → 자동 압축(auto-compact)이 앞부분을 요약으로 대체
  → 변수명은 일반화되고, 스택 트레이스와 "왜 그렇게 결정했는지"가 사라짐
  → 에이전트는 요약만 보고 이미 고친 것을 다시 고치거나, 정한 것을 뒤집음
```

증상

- 아까 통과한 테스트가 다시 깨진다
- 이미 결정한 구조를 새로 제안한다
- 같은 실패를 처음 보는 것처럼 반복한다

실제 사례 — Issue #22758 (2026-02)

밤새 켜둔 세션에서 압축이 타임아웃 > 재시도 루프에 빠짐. **종료 조건이 없어서** 서브에이전트 27개 생성, 2.5시간, API 요청 4,246건. 사람은 아무것도 시키지 않았습니다.

압축은 버그가 아니라 기본 동작입니다. **대화 안에만 있는 기억은 언젠가 요약됩니다.** 그래서 기억을 대화 밖에 두고, 반복에 상한을 둡니다.

anthropics/claude-code#22758 — "Auto-compact enters infinite retry loop after timeout, consuming rate limit"

루프의 척추는 대화 밖의 기억입니다

```
1. 이번 실행이 끝났다면
2. SPEC + 코드·테스트 + MEMORY 를 갱신하고
3. 다음 실행이 현재 상태와 다음 행동을 읽는다
```

기억할 것	저장할 곳
현재 구현 상태	코드와 테스트
목표와 완료 조건	`SPEC_NN.md`
진행 상태·반복 횟수·시도와 실패	`MEMORY.md`

- `MEMORY.md` 는 **에이전트만 씁니다.** 사람은 읽고, 판정은 **다음 프롬프트로** 전달합니다.
- "완료했다"는 문장보다 **검사 결과와 재현 가능한 근거**를 저장합니다.

모델의 대화는 끝나도, 저장소의 상태는 다음 실행으로 이어집니다.

이미 있는 것과 뭐가 다를까요

Jenkins·GitHub Actions·Airflow도 "실행하고 > 검증하고 > 다음을 정합니다"

구분	워크플로 엔진 (CI/CD, Airflow, Temporal)	그래프 엔지니어링	루프 엔지니어링
경로를 언제 정하나	**작성할 때** — YAML·DAG에 고정	**작성할 때** — 의존 관계만 선언	**실행하면서** — 결과를 보고 다음을 고름
각 단계 안에서	정해진 명령 실행	정해진 작업, 병렬·부분 재실행 가능	에이전트가 판단하며 도구 사용
실패하면	정해진 재시도 후 사람 호출	실패한 노드와 하류만 다시	원인을 분류해 코드·스펙·환경 중 고칠 곳 결정
상태는 어디에	엔진의 로그·아티팩트	그래프 실행기	저장소의 파일 (`MEMORY.md`)
오늘 실습에서	—	SPEC 머리말의 `depends_on` 이 씨앗	오전 내내 도는 것

경로를 미리 그릴 수 있으면 워크플로·그래프, 실행하면서 찾아야 하면 루프, 그것이 돌아갈 환경이 하네스입니다. 셋은 경쟁하지 않습니다 — 루프는 대개 CI 안에서 한 단계로 돌아갑니다.

네 이름은 서로 대체하지 않고 쌓입니다

구분	설계하는 것	핵심 질문
프롬프트 엔지니어링	모델에 전달할 지시와 맥락	이번 작업을 무엇으로, 어떤 형식으로 받을까?
하네스 엔지니어링	에이전트가 일하는 실행 체계	무엇을 읽고 실행하며, 어떻게 제한·검증할까?
루프 엔지니어링	여러 실행을 잇는 외부 업무 흐름	언제 시작하고, 다음에는 무엇을 하며, 언제 멈출까?
그래프 엔지니어링	작업 사이의 의존 관계	무엇이 끝나야 무엇을 시작할 수 있을까?

> 프롬프트는 **한 번의 시도**를 설명하고, 하네스는 그 시도를 **일관된 조건**에서 실행하며, 루프는 결과를 **다음 시도의 입력**으로 바꾸고, 그래프는 시도들의 **순서와 의존**을 선언합니다.

상위 층은 아래층을 없애지 않습니다. **좋은 루프 안에는 좋은 하네스와 좋은 프롬프트가 함께 있습니다.**

운영형 루프를 키우는 다섯 장치

확장 장치	더해지는 능력	안 쓰면 생기는 일	오늘의 필수 실습
자동 실행	일정·이벤트에 맞춰 작업 시작	사람이 시작 버튼을 계속 누른다	사용하지 않음 — 사람이 시작
작업 격리	병렬 작업끼리 파일 충돌 방지	동시에 돌린 작업이 서로의 파일을 덮어쓴다	사용하지 않음 — 한 폴더·한 SPEC
Skill	반복되는 프로젝트 지식 재사용	같은 규칙을 프로젝트마다 다시 쓴다	직접 만드는 커맨드로 체험
플러그인·커넥터	외부 도구와 실제 업무 시스템에 연결	루프가 실제 업무 시스템에 닿지 못한다	Ralph만 선택 실습
서브에이전트	만드는 역할과 검사하는 역할 분리	만든 쪽이 자기 결과를 검사한다	사람이 독립 검증자 역할

이 장치들은 첫 루프의 필수 조건이 아닙니다. **수동 루프가 안정된 뒤 필요한 능력만 추가합니다.** Osmani가 그린 해부도도 같은 다섯 가지에 외부 상태를 더한 것입니다.

Osmani, Loop Engineering (2026-06): automations · worktrees · skills · plugins/connectors · sub-agents · external state

오늘의 테트리스는 외부 루프의 축소판입니다

```
SPEC 00  골격과 화면
   ↓
SPEC 01  블록 이동과 회전
   ↓
SPEC 02  충돌, 줄 제거, 게임오버
   ↓
SPEC 03  점수와 리더보드
   ↓
SPEC 04  레벨과 편의 기능
   ↓
SPEC 05+ 선택한 게임 기능
```

단계마다 반복할 네 질문

- 이번 SPEC의 완료 조건은 무엇인가?
- 무엇으로 통과를 증명할 것인가?
- 실패했다면 코드·스펙·환경 중 어디를 고칠 것인가?
- 다음으로 갈 수 있는가, 멈춰야 하는가?

기능은 단계마다 늘어나지만 **루프의 모양은 한 번도 바뀌지 않습니다.** 한 번에 SPEC 하나만 다루고, 검증 결과에 따라 다음 상태를 결정합니다.

좋은 루프와, 잘 돌아가던 나쁜 루프

좋은 루프

- 입력과 출력이 분명합니다.
- 완료 조건을 확인할 수 있습니다.
- 작업자와 검증 기준이 분리되어 있습니다.
- 상태와 근거가 대화 밖에 남습니다.
- 반복 상한과 사람에게 넘길 조건이 있습니다.

넉 달 뒤에 무너진 루프

Dex Horthy (HumanLayer)는 사람이 코드를 한 줄도 읽지 않는 **완전 자동 소프트웨어 공장**을 약 넉 달 돌렸습니다.

- 테스트는 계속 초록이었습니다.
- 코드는 계속 늘었습니다.
- 그런데 아무도 그 코드를 이해하지 못했고, 버그 하나를 잡는 데 몇 주가 걸리기 시작했습니다.

그가 붙인 이름 — **이해 부채**(comprehension debt): 존재하는 코드의 양과 사람이 아직 이해하는 양 사이의 벌어지는 간격.

검증이 통과한다는 것과 사람이 책임질 수 있다는 것은 다릅니다. 좋은 루프의 다섯 조건은 이 간격을 벌리지 않기 위한 것입니다.

Horthy, Harness Engineering is not Enough: Why Software Factories Fail (AI Engineer World's Fair, 2026) · Osmani 정리, Software Factories, Light and Dark (2026-07)

루프를 만들되, 엔지니어로 남습니다

검증

- “완료”는 주장이지 증명이 아닙니다.
- 자동 검사와 직접 재현으로 확인합니다.

이해

- 바뀐 코드와 판단 근거를 읽습니다.
- 이해하지 못한 결과를 그대로 넘기지 않습니다.

권한

- **사람 승인**: 외부 발송·결제·운영 변경
- **자동화 금지**: 민감 정보·고위험 판단

루프를 쓰지 말아야 하는 일 — 디자인·제품 판단이 필요한 일, 요구사항이 아직 애매한 일, 되돌릴 수 없는 운영 변경. 이런 일은 루프가 아니라 **대화**로 합니다.

첫 루프의 기본값은 `초안 + 근거 자료 + 사람 검토`**, 반복에는 반드시 상한을 둡니다.**

02

Claude Code 준비하기

코드보다 하네스를 먼저 만들기

Claude Code 설치하기

macOS / Linux / WSL

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

사내 정책이 원격 스크립트 실행을 막으면 npm으로 설치합니다.

```
npm install -g @anthropic-ai/claude-code
```

설치 후 실습 폴더에서 `claude`를 실행하고, 브라우저가 열리면 사용할 계정으로 로그인합니다. 둘 다 막히면 Claude Code 공식 설치 문서를 확인합니다.

Claude Code 안에서도 역할을 나눕니다

역할	담당	하는 일
스펙 검토·계획	Claude Code Plan mode	저장소를 읽고 애매함을 찾되 파일은 수정하지 않음
구현·실행	Claude Code	승인한 계획과 스펙 범위 안에서 수정하고 검사함
결과 관찰	우리	게임을 직접 조작하고 기대 결과와 비교함
판정 전달	우리	관찰한 것을 다음 실행의 프롬프트로 넘김
상태 기록	`MEMORY.md`	에이전트가 통과·실패의 근거와 다음 결정을 남김

- Plan mode에서는 구현 전에 저장소를 탐색하고 계획을 검토합니다.
- 계획을 승인한 뒤 실행하되, 최종 판정은 검증 결과를 본 사람이 합니다.
- 사람은 파일을 손으로 쓰지 않습니다. **판정은 프롬프트가 되고, 기록은 에이전트가 남깁니다.**

실습 0. 코드보다 하네스를 먼저 만듭니다

첫 프롬프트로 테트리스를 시키지 않습니다

- 오늘 우리는 SPEC 한 장을 최대 세 번씩, 여러 단계에 걸쳐 **열 번 넘게 실행**합니다.
- 매번 "폴더 밖은 건드리지 마", "검증부터 확인해", "한 번만 실행해"를 다시 쓴다면 그건 부탁입니다.
- 같은 부탁을 **파일로 한 번 적어두면 규칙**이 됩니다. 그것이 하네스입니다.
- 그래서 첫 실행은 게임이 아니라 **일하는 방식**을 만드는 데 씁니다.

> 재료는 이미 있습니다. `SPEC_00_PROJECT.md` 한 장에 기술 제약, 폴더 경계, 반복 상한, 검증 방법이 모두 적혀 있습니다.

하네스는 처음부터 완벽할 필요가 없습니다. 오늘 겪는 실패가 내일의 규칙이 됩니다.

하네스 설계 프롬프트

빈 `tetris-loop-workshop` 폴더에서 `claude` 를 실행하고, `/plan` 또는 `Shift+Tab` 으로 Plan mode에 들어간 뒤 요청합니다.

```
@SPEC_00_PROJECT.md 를 읽어줘.

이 문서를 여러 번 반복 실행할 워크숍의 하네스를 먼저 만들어줘.
지금 테트리스 코드는 만들지 마.

1. CLAUDE.md — 기술 제약, 폴더 경계, 정지 조건, 완료 선언의 근거 규칙
2. .claude/commands/spec-new.md — 요구사항을 SPEC 초안으로 바꾸는 커맨드
3. .claude/commands/spec-run.md — SPEC 한 장을 실행하고 검증·보고하는 커맨드
4. MEMORY.md — 다음 실행이 읽을 현재 상태와 반복된 실패
   (에이전트만 갱신한다. 내가 직접 확인한 결과는 프롬프트로 전달할 테니
    그것도 함께 기록해줘)

먼저 계획을 보여주고, 내가 승인하면 파일을 만들어줘.
```

각 파일에 들어갈 조건까지 적은 전문은 실습 자료에 있습니다.

하네스가 만들어야 하는 네 가지

산출물	하네스에서의 역할	반드시 들어갈 것
`CLAUDE.md`	지시와 경계	번들러·패키지·개발 서버 없음, 폴더 밖 수정 금지, 같은 실패 2회·한 SPEC 3회 상한
`/spec-new`	스펙을 만드는 루프	완료 조건은 `입력 → 기대 결과` 형식, 애매한 항목은 `[사람 확인 필요]`
`/spec-run`	스펙을 실행하는 루프	선행 SPEC 유지, 검증 먼저, 최소 변경, 한 번만 실행, 완료 조건별 통과 보고
`MEMORY.md`	실행 사이의 기억	통과한 SPEC, 진행 중인 SPEC과 실행 횟수, 반복된 실패, 사람이 전달한 판정

네 파일이 아침에 만든 하네스의 전부입니다. **오후 내내 이 네 파일이 일하고, 우리는 SPEC과 판정만 넣습니다.**

만들어진 하네스를 검수합니다

승인하기 전에 사람이 직접 읽습니다

- ☐ `CLAUDE.md` 의 기술 제약이 `SPEC_00_PROJECT.md` 와 같은가
- ☐ 작업 경계(폴더 밖 수정 금지, 외부 배포 금지)가 문장으로 적혀 있는가
- ☐ 정지 조건에 **숫자**가 있는가 — "적당히"가 아니라 `2회` , `3회`
- ☐ 완료 선언에 검사 결과와 재현 절차를 요구하는가
- ☐ `/spec-run` 이 **SPEC 한 장만** 다루고, 한 번만 실행하는가
- ☐ `MEMORY.md` 에 다음 실행이 읽을 항목이 정해져 있는가
- ☐ 두 커맨드가 `.claude/commands/` 아래에 있는가 — 다른 위치에 만들어졌다면 슬래시 커맨드로 뜨지 않습니다

빠진 것이 보이면 파일을 직접 고치거나, 무엇이 빠졌는지 말해 다시 만들게 합니다.

하네스를 읽지 않고 승인하면, 오늘 하루를 읽지 않은 규칙에 맡기는 것입니다.

Claude Code Plan mode · Claude Code 권장 작업 방식

필수 실습은 사람이 직접 루프를 돌립니다

실행은 커맨드로, 시작은 사람이

```
/spec-run SPEC_NN.md
```

- 매 반복의 **시작 버튼은 사람이 누릅니다.** 자동 반복은 오늘 필수 실습에 없습니다.
- 한 번 실행하고, 자동 검사와 직접 플레이로 확인하고, `MEMORY.md`가 갱신됐는지 읽습니다.
- 통과하면 다음 스펙으로, 실패하면 관찰한 것을 다음 프롬프트에 담아 현재 단계만 보완합니다.

처음부터 자동으로 반복시키면 **무엇 때문에 좋아졌는지 관찰할 수 없습니다.**

SPEC 00~03에서는 한 스펙당 최대 세 번만 실행합니다.

선택 도구: Ralph Wiggum 루프

SPEC 04처럼 완료 조건과 자동 검사가 준비된 뒤에만 사용합니다.

```
/plugin install ralph-wiggum@claude-plugins-official

/ralph-loop "SPEC_04_LEVEL.md를 구현하고 모든 검증을 실행해.
완료되면 SPEC_04_COMPLETE를 출력해." \
  --completion-promise "SPEC_04_COMPLETE" \
  --max-iterations 3
```

- 같은 프롬프트를 다시 전달하며 결과를 반복 개선하는 Claude Code 공식 플러그인입니다.
- `--max-iterations`는 반드시 지정합니다. 기본값인 무제한 반복을 사용하지 않습니다.
- 디자인 판단, 애매한 요구사항, 사람의 승인이 필요한 작업에는 사용하지 않습니다.
- **사내망에서 `/plugin install`이 되는지 교육 전날 확인합니다.** 안 되면 SPEC 04도 `/spec-run`으로 수동 반복합니다.

Ralph Wiggum 공식 문서

실습 준비 완료 체크

- 실습 자료 전체 열기
- ☐ `tetris-loop-workshop` 빈 폴더를 만들었다.
- ☐ 실습 자료의 마크다운 파일을 폴더에 복사했다.
- ☐ 터미널에서 `claude`를 실행하고 로그인했다.
- ☐ Plan mode로 하네스 설계를 요청하고 계획을 검수했다.
- ☐ `CLAUDE.md`, `.claude/commands/spec-new.md`, `.claude/commands/spec-run.md`, `MEMORY.md`가 만들어졌다.
- ☐ 브라우저에서 `index.html`을 직접 열 수 있다.
- ☐ 외부 배포 없이 로컬 프로젝트 안에서만 작업한다.

안 되면 — 설치가 막히면 npm 경로로 · 브라우저 로그인이 안 되면 터미널에 표시된 코드로 · 그래도 안 되면 채팅으로 알리고, 강사 화면 공유로 실습 0을 먼저 따라갑니다.

Ralph 플러그인은 필수가 아닙니다. **하네스를 만들고 수동 루프를 제대로 기록하는 것이 오늘의 필수 목표**입니다.

03

마크다운 스펙으로 테트리스 루프 설계하기

아이디어를 실행하고 검증할 수 있는 명세로 바꾸기

오늘의 실습은 하나입니다

빈 폴더에서 시작해, 스펙을 한 장씩 추가하며 테트리스를 발전시킵니다

스펙	추가되는 기능	이번 단계에서 우리가 하는 일
SPEC 00	프로젝트 골격, 보드, 상태 패널	완성된 스펙을 읽고 애매한 표현 표시
SPEC 01	블록 생성, 낙하, 이동, 회전	완료 조건과 검증 시나리오 보완
SPEC 02	충돌, 고정, 줄 제거, 게임오버	예외 조건을 포함한 전체 스펙 완성
SPEC 03	점수, 하드 드롭, 로컬 리더보드	저장 규칙까지 검증 가능한 스펙 작성
SPEC 04	레벨, 속도, 다음 블록, 일시정지	기존 기능을 지키는 확장 스펙 설계
선택	A 게임 기능 확장 / B 내 업무 루프	범위·완료 조건·검증까지 직접 설계

완성된 프로젝트나 코드 골격은 제공하지 않습니다. 처음에는 좋은 스펙을 따라 하고, 마지막에는 **스스로 다음 루프를 설계**합니다.

왜 테트리스일까요?

루프에 유리한 점

- 입력과 상태 변화가 눈에 보입니다.
- 성공과 실패 조건이 분명합니다.
- 기능을 작은 단계로 나누기 쉽습니다.
- 이전 기능이 깨졌는지 바로 확인할 수 있습니다.

우리가 집중할 것

- 코드를 얼마나 많이 만들었는가 ❌
- AI가 완료했다고 말했는가 ❌
- 스펙의 조건을 재현할 수 있는가 ✅
- 실패 원인을 코드·현재 SPEC·환경 중 맞는 곳에 반영했는가 ✅

스펙 시트는 한 루프의 명세서입니다

- 무엇을 만들지뿐 아니라 **언제 끝났다고 판단할지** 적습니다.
- 화면, 입력, 상태 변화, 예외를 코드보다 먼저 결정합니다.
- AI가 추측해야 하는 빈칸은 `[사람 확인 필요]`로 남깁니다.
- 구현과 스펙이 충돌하면 코드를 임의로 고치기 전에 스펙을 먼저 확인합니다.
- 하나의 스펙은 최대 세 번 안에 검증할 수 있는 크기로 제한합니다.

프롬프트는 요청이고, 스펙은 반복해서 읽을 수 있는 기준입니다.

모든 스펙은 같은 구조를 사용합니다

항목	답해야 하는 질문
목표와 선행 스펙	무엇을 추가하고, 무엇이 이미 동작해야 하는가?
화면 정의	어떤 정보와 상태가 어디에 표시되는가?
기능 규칙	입력을 받으면 상태가 어떻게 변하는가?
예외 및 금지 조건	무엇을 거부하고, 무엇은 하지 않는가?
완료 조건	어떤 결과를 재현하면 끝인가?
검증 시나리오	성공과 실패를 어떻게 확인하는가?
반복·중단 조건	몇 번 다시 하고, 언제 사람에게 넘기는가?

이 양식은 게임이 아니라 업무 스펙에도 그대로 사용합니다.

첫 번째 스펙: SPEC 00

```
# SPEC 00 — 프로젝트 골격과 게임 화면

## 목표

브라우저에서 실행되는 테트리스 프로젝트 골격을 만든다.

## 화면 정의

- 왼쪽: 10×20 게임 보드
- 오른쪽: 점수, 제거한 줄, 게임 상태
- 하단: 시작/재시작 버튼과 조작 안내

## 완료 조건

- `index.html`을 브라우저에서 직접 열면 콘솔 오류가 없다.
- `test.html`의 자동 검사가 모두 통과한다.
- 보드의 셀은 10열×20행이다.
- 아직 블록은 움직이지 않는다.
```

완성본은 실습 자료의 `SPEC_00_PROJECT.md`에서 확인합니다.

실습 1. 스펙을 실행하기 전에 먼저 검토합니다

`SPEC_00_PROJECT.md`를 읽고 아래 항목에 표시합니다.

- 서로 다르게 해석할 수 있는 표현이 있는가
- 화면에 표시할 위치와 상태가 분명한가
- 완료 조건을 직접 재현할 수 있는가
- 프로젝트 폴더 밖 수정처럼 금지할 행동이 적혀 있는가
- 실패했을 때 몇 번까지 반복할지 적혀 있는가

> **Claude Code의 Plan mode에서 이렇게 요청해보세요**
> SPEC_00_PROJECT.md를 검토해줘. 구현하지 말고, 서로 다르게 해석할 수 있는 부분, 빠진 예외, 검증할 수 없는 완료 조건만 표로 정리해줘.

좋은 완료 조건은 눈으로 다시 확인할 수 있습니다

나쁜 완료 조건	좋은 완료 조건
테트리스처럼 예쁘게 만든다	게임 보드는 10열×20행으로 표시된다
키보드가 자연스럽게 동작한다	`←`, `→`, `↓`, `↑` 입력 결과를 각각 재현한다
점수가 잘 계산된다	한 줄 제거 시 100점, 네 줄 제거 시 800점이다
리더보드가 잘 저장된다	새로고침 후에도 상위 10개 기록이 유지된다

완료 조건에는 가능하면 **입력 > 기대 결과**를 함께 씁니다.

실습 2. 다음 기능을 스펙으로 바꿉니다

아래 요구사항을 `SPEC_01_MOVE.md` 의 완료 조건과 검증 시나리오로 바꿔봅니다.

> 블록은 자동으로 내려온다. 방향키로 좌우 이동, 아래 이동, 회전을 할 수 있다. 보드 밖으로는 나가지 않는다.

반드시 포함할 것:

- 네 개의 키 입력과 각각의 기대 결과
- 벽 옆에서 이동·회전할 때의 처리
- 아직 SPEC 01에서 구현하지 않을 기능
- 통과 여부를 확인할 자동 검사와 수동 검사

요구사항을 길게 쓰는 것이 아니라, AI와 사람이 같은 결과를 기대하게 만드는 것이 목표입니다.

04

검증 방법과 기록 준비하기

"잘 됐다"의 근거를 남기고, 멈춰야 할 순간 정하기

검증은 세 겹으로 만듭니다

검증 층	확인할 것	예시
구조 검증	파일과 실행이 정상인가	`test.html` 자동 검사, `index.html` 콘솔
규칙 검증	입력에 따른 상태 변화가 스펙과 같은가	한 줄 제거 시 100점
경험 검증	실제 화면과 조작이 의도대로 느껴지는가	방향키, 게임오버, 리더보드 직접 확인

자동 검사는 빠르고 반복하기 좋지만, 화면과 사용자 경험을 전부 판단하지는 못합니다.

자동 검사와 수동 검사의 결과가 모두 있어야 해당 스펙을 완료합니다. AI의 마지막 답변은 기준이 아닙니다.

판정은 프롬프트로, 기록은 `MEMORY.md`로

실행 전 · 사람

- 이번에 돌릴 SPEC
- 기대하는 결과
- 이번 반복에서 하지 않을 일

실행 직후 · 에이전트

- 생성·수정한 파일
- 자동 검사 결과
- 실패 메시지 원문
- 지금까지의 실행 횟수

직접 검증 후 · 사람

- 재현한 시나리오와 실제 결과
- `통과` / `실패` / `사람 확인 필요`
- 스펙에서 보완할 내용
- **다음 프롬프트에 담아 전달**

좋았다는 감상 대신, 다른 사람도 같은 결과를 확인할 수 있는 근거를 남깁니다.

예시: SPEC 01 첫 번째 실행

에이전트가 `MEMORY.md` 에 남긴 것

- 진행 중: SPEC 01, 1회차
- 수정: `game.js` , `main.js` , `test.js`
- 자동 검사: 12개 통과, 회전 경계 1개 실패
- 실패 원문: `expected x=8, got x=9`
- 미해결: 회전이 벽을 넘을 때의 처리

사람이 다음 프롬프트로 보낸 것

> 오른쪽 벽에서 `↑` 를 누르면 블록 일부가 보드 밖으로 나가. `SPEC_01` 에 "회전 결과가 경계를 벗어나면 회전을 거부하고 기존 방향을 유지한다"를 추가하고 두 번째 실행을 해줘. 이번 관찰도 `MEMORY.md` 에 남겨줘.

"회전이 이상함"이 아니라 **어떤 상태에서 어떤 입력을 주니 무엇이 달랐는지**를 전달합니다.

사람의 판정은 파일이 아니라 다음 실행의 입력이 됩니다.

루프는 멈출 줄 알아야 합니다

멈추는 조건

- 같은 실패가 두 번 반복됨
- 한 스펙의 총 실행 횟수가 세 번을 넘음
- 스펙끼리 서로 충돌함
- 화면이나 규칙에 새로운 제품 판단이 필요함
- 프로젝트 폴더 밖 수정이나 외부 배포가 필요함
- 필요한 입력 또는 실행 환경이 없음

세 번을 다 썼다면

1. 통과한 완료 조건만 `MEMORY.md` 에 남기고, 못 한 조건은 **범위 밖으로 옮겨 SPEC을 축소**합니다. 축소한 SPEC은 한 번 더 실행할 수 있습니다.
2. 그래도 안 되면 **다음 SPEC으로 가지 않습니다.** 남은 시간은 `MEMORY.md` 를 읽으며 어디서 같은 실패가 반복됐는지 분석하는 데 씁니다.
3. 실제 업무에서도 순서는 같습니다 — 범위 축소 > 사람에게 이관. **이것도 루프의 정상 종료입니다.**

중단할 때는 `진행 불가` 만 남기지 말고, **시도한 것·실패 근거·필요한 결정**을 `MEMORY.md` 에 남기게 합니다.

스펙별 회귀 검증

새 스펙을 통과해도 이전 기능이 깨졌다면 완료가 아닙니다.

현재 단계	새로 확인할 것	다시 확인할 것
SPEC 00	골격, 화면, 자동 검사	-
SPEC 01	이동, 낙하, 회전	SPEC 00 화면과 초기값
SPEC 02	충돌, 고정, 줄 제거, 게임오버	SPEC 00~01
SPEC 03	점수, 하드 드롭, 리더보드	SPEC 00~02
SPEC 04	레벨, 속도, 다음 블록, 일시정지	SPEC 00~03 전체
선택 A	확장한 게임 기능	SPEC 00~04 전체
선택 B	업무 SPEC의 완료 조건	하네스 규칙 준수

`MEMORY.md` 의 회귀 검증 항목에서 이전 SPEC까지 모두 통과해야 다음 단계로 이동합니다.

05

스펙을 실행하고,
테트리스를 발전시키기

SPEC 00 > 01 > 02 > 03 · 선택 04 > 05+

한 번의 루프

```
스펙 선택
  → Claude Code Plan mode로 애매함과 실행 계획 확인
  → 계획을 승인하고 /spec-run 딱 한 번 실행
  → test.html 자동 검사
  → index.html에서 검증 시나리오 재현
  → MEMORY.md 갱신 확인
  → 통과 / 원인에 맞게 보완 / 중단
```

- 한 번에 `SPEC_NN.md` 하나만 다룹니다.
- 실패했다면 먼저 원인을 관찰하고, 필요할 때 스펙을 고칩니다.
- 통과하기 전에는 다음 SPEC을 섞지 않습니다.

실행은 우리가 만든 커맨드로 합니다

```
/spec-run SPEC_NN.md
```

아침에 만든 `/spec-run`에는 이미 이 규칙들이 들어 있습니다.

- 선행 SPEC의 완료 조건을 유지한다.
- 프로젝트 폴더 밖은 수정하지 않는다.
- 먼저 실패하는 검사를 `test.js`에 추가하거나 기존 검증을 확인한다.
- 완료 조건에 필요한 최소한의 코드만 변경한다.
- `test.html`의 자동 검사와 `index.html`의 콘솔 오류를 확인한다.
- 완료 조건별 통과 여부와 사람이 직접 확인할 시나리오를 보고한다.
- 애매한 요구사항은 추측하지 않고 `[사람 확인 필요]`로 알린다.
- 이번 실행은 한 번만 수행한다.

바뀌는 것은 커맨드가 아니라 **입력하는 SPEC과 `MEMORY.md`에 쌓인 상태**입니다.

프로젝트 골격

SPEC 00 · 정적 화면

기능과 화면

- HTML5 + CSS + 브라우저 기본 JavaScript, 빌드 단계 없음
- `index.html`, `game.js`, `main.js`, `style.css`, `test.html`, `test.js`
- 브라우저에서 `index.html`과 `test.html`을 직접 열어 확인
- 제목 `TETRIS LOOP`, 10열×20행 보드
- 점수 `0`, 제거한 줄 `0`, 상태 `READY` 패널
- `시작` 버튼과 전체 키 조작 안내
- 좁은 화면에서는 세로 배치, 가로 스크롤 없음

상태와 범위

- `globalThis.TetrisGame`으로 공개
- `BOARD_WIDTH=10`, `BOARD_HEIGHT=20`
- `GAME_STATUS`: `READY`, `PLAYING`, `LANDED`, `PAUSED`, `GAME_OVER`
- `createEmptyBoard()`: 독립된 20×10의 `0` 배열
- `createInitialState()`: 빈 보드와 초기 표시값
- **범위 밖**: 블록 생성·입력·타이머·충돌·점수·저장

SPEC 00은 움직이는 게임이 아니라 **이후 SPEC이 의존할 실행 가능한 골격과 초기 상태**를 고정합니다.

SPEC 00 — 완료 조건과 검증

자동 검증

- 보드 상수는 각각 `10`, `20` 이다.
- 빈 보드는 20×10이고 모든 셀이 `0` 이다.
- 한 행을 바꿔도 다른 행은 바뀌지 않는다.
- 초기 상태는 빈 보드·점수 `0` ·줄 `0` · `READY` 다.
- `test.html` 의 모든 검사가 통과한다.

직접 검증

- 보드·상태 패널·시작 버튼·키 안내가 모두 보인다.
- DOM의 보드 셀은 정확히 200개다.
- 화면 폭 390px에서도 가로 스크롤이 없다.
- 기다리거나 방향키를 눌러도 상태가 바뀌지 않는다.
- `index.html` 을 직접 열었을 때 콘솔 오류가 없다.

완료 판정: 자동 검사와 직접 검증이 모두 통과하고, 범위 밖 기능이 구현되지 않았다.

SPEC 01 — 생성·이동·회전

생성과 입력

- `시작` : `READY → PLAYING`, 블록 하나 생성
- I·O·T·S·Z·J·L의 표준 모양과 구분 가능한 색
- 블록 너비 기준 보드 상단 중앙에 생성
- `PLAYING`에서는 700ms마다 한 셀 자동 낙하
- `←` · `→` · `↓` : 한 셀 이동, `↑` : 시계 방향 90° 회전
- 방향키의 브라우저 기본 스크롤 방지

경계와 상태

- 좌·우·아래 경계를 넘는 이동은 이전 상태 유지
- wall kick 없음: 경계를 넘는 회전은 거부
- 더 내려갈 수 없으면 `LANDED`, 낙하 타이머 정지
- `LANDED`에서는 방향키 무시, 버튼은 `재시작`
- 재시작하면 빈 보드·새 블록·타이머 하나로 초기화
- **범위 밖:** 고정 블록 충돌·줄 제거·점수·하드 드롭

무작위 생성기는 주입 가능하게 만들어, 검사에서는 원하는 블록을 선택할 수 있어야 합니다.

SPEC 01 — 완료 조건과 검증

자동 검증	준비 > 입력 > 기대 결과
블록 정의·생성	지정한 일곱 모양이 상단 중앙 좌표에 생성된다.
이동·회전	열린 공간에서 좌표가 한 셀 변하고, 회전 행렬은 시계 방향 90° 결과다.
경계 거부	벽·바닥을 넘는 이동과 회전은 위치·방향을 바꾸지 않는다.
낙하·착지	가짜 타이머의 한 tick은 `y+1`, 바닥 tick은 `LANDED`와 타이머 정지를 만든다.
재시작	보드·블록·상태를 초기화하며 활성 낙하 타이머는 하나다.

직접 검증 시나리오

1. 시작 직후 블록 하나가 상단 중앙에 보인다.
2. 자동 낙하와 `↓` 입력이 각각 한 셀만 이동시킨다.
3. 양쪽 벽에서 추가 이동과 불가능한 회전을 해도 상태가 유지된다.
4. 바닥 도착 후 방향키가 무시되고, 재시작하면 빈 보드로 돌아간다.
5. 게임 중 방향키를 눌러도 페이지가 스크롤되지 않는다.

SPEC 01·02에서 자주 막히는 곳

증상	원인	확인법
벽 옆에서 `↑`를 누르면 블록이 밖으로 나감	회전 결과에 경계 검사를 하지 않음	오른쪽 벽에 붙이고 `↑` — 일부 셀이 사라지는가
재시작하면 낙하가 두 배 빨라짐	기존 타이머를 지우지 않고 새로 만듦	재시작 세 번 뒤 낙하 속도를 처음과 비교
한 줄을 지우면 다른 줄도 바뀜	`Array(20).fill(row)`로 모든 행이 같은 배열 참조	빈 보드 두 개의 첫 행이 `===`로 같은가
방향키를 누르면 페이지가 스크롤됨	키 이벤트에 `preventDefault` 누락	세로로 긴 창에서 `↓` — 화면이 움직이는가

막히면 이 표부터 봅니다. 여기 없으면 `MEMORY.md`의 **실패 메시지 원문**을 들고 강사에게 옵니다.

흔한 실패 자료 열기

`MEMORY.md` 중간 점검

현재 세션을 닫고 **새 세션**을 엽니다. 새 세션은 대화를 기억하지 못하므로, `MEMORY.md` 만 읽고 다음 질문에 답하게 합니다.

> `MEMORY.md` 만 읽고 답해줘. 지금 어느 SPEC의 몇 번째 실행까지 왔고, 마지막 실패는 무엇이었고, 다음에 무엇을 해야 해?

새 세션의 답을 내 기억과 비교합니다.

- 실패한 입력과 기대 결과를 재현할 수 있는가
- "잘 됨", "이상함" 같은 판정만 남아 있지 않은가
- 코드 수정 내용이 아니라 스펙의 빈틈이 기록되어 있는가
- 내가 프롬프트로 전달한 관찰이 기록에 반영되었는가
- 같은 실패를 두 번 반복했다면 멈췄는가

기억이 없는 세션이 기록만 보고 이어받을 수 있다면, 그 기록은 루프의 척추 역할을 하고 있는 것입니다.

SPEC 02

충돌·고정·줄 제거·게임오버 규칙

핵심 규칙

- 움직이는 블록과 고정된 보드를 별도 상태로 관리
- 고정 블록과 겹치는 이동·회전은 이전 상태 유지
- 여러 완성 줄은 한 번에 제거하고 위 행을 압축
- 다음 블록의 생성 위치가 막히면 `GAME_OVER`
- 게임오버에서는 타이머 정지, 키 입력과 예약 tick 무시
- 재시작은 빈 보드·점수 `0`·줄 `0`·타이머 하나로 초기화

범위 밖: 점수 계산·하드 드롭·리더보드·레벨·다음 블록 표시·일시정지

아래로 갈 수 없을 때의 순서

```
현재 블록을 보드에 고정
  → 완성된 모든 줄 탐색
  → 줄 동시 제거
  → 위쪽 행 압축
  → 제거한 줄 수 누적
  → 다음 블록 생성
```

한 입력 또는 tick에서 이 처리는 **정확히 한 번**만 실행합니다.

SPEC 02 — 완료 조건과 검증

자동 검증: 보드 fixture

- 블록 고정 후 지정 좌표만 보드에 병합된다.
- 고정 셀과 겹치는 이동·회전은 전체 상태를 유지한다.
- 한 줄·여러 줄 제거 후 빈 행 수와 누적 줄 수가 맞다.
- 남은 행의 순서가 유지되고 새 빈 행은 참조를 공유하지 않는다.
- 생성 위치 충돌은 `GAME_OVER` 와 타이머 제거를 만든다.
- 연속 재시작에도 활성 낙하 타이머는 하나다.

직접 검증: 재현 시나리오

- 한 줄·두 줄 완성 직전 fixture에서 줄 제거를 실행한다.
- 고정 블록 옆에서 이동·회전을 시도한다.
- 생성 영역을 채운 fixture에서 게임오버를 확인한다.
- 게임오버 뒤 방향키를 눌러도 상태가 바뀌지 않는다.
- 재시작을 반복해도 낙하 속도가 빨라지지 않는다.
- SPEC 03 전이므로 줄 제거 후에도 점수는 `0` 이다.

회귀: SPEC 00~01 전체 조건과 `test.html` 의 모든 검사가 통과해야 한다.

점수와 하드 드롭

SPEC 03 · 행동을 숫자로 기록하기

행동	점수
한 줄 제거	100점
두 줄 동시 제거	300점
세 줄 동시 제거	500점
네 줄 동시 제거	800점
소프트 드롭 `↓`	직접 내린 셀당 1점
하드 드롭 `Space`	즉시 내린 셀당 2점

- 현재 점수와 제거한 줄 수를 즉시 표시하고, 재시작하면 둘만 `0`으로 초기화합니다.
- 자동 낙하와 이동하지 못한 `↓`에는 점수를 주지 않습니다.
- `Space`는 충돌 직전까지 이동해 실제 이동 셀당 2점을 더하고 고정을 한 번 실행합니다.
- 드롭과 줄 제거가 함께 발생하면 더하되, 이 단계에서는 레벨 배수를 적용하지 않습니다.

SPEC 03 — 로컬 리더보드

한 번의 게임 결과를 다음 실행에도 남깁니다

- 게임오버 시 최종 점수·줄 수, 이름 입력, 저장 버튼을 표시합니다.
- 이름은 trim 후 보이는 문자 기준 2~10자이며 한글·영문·숫자만 허용합니다.
- 유효하지 않으면 이유를 입력란 옆에 표시하고 저장하지 않습니다.
- 한 게임의 기록은 한 번만 저장합니다.
- 저장 키는 `tetris-loop.leaderboard.v1` 입니다.
- 기록은 `id`, `name`, `score`, `clearedLines`, `playedAt`을 가집니다.
- 점수 내림차순 > 이른 `playedAt` > 기존 순서로 정렬합니다.
- 상위 10개만 저장하며 손상된 JSON은 빈 목록으로 복구합니다.
- 초기화는 사용자 확인 후에만 실행합니다.

범위 밖: 서버·온라인 순위·로그인·여러 기기 동기화

SPEC 03

완료 조건과 검증

검증 영역	자동 검사할 규칙	브라우저에서 직접 확인할 것
점수	1·2·3·4줄 점수, 성공한 소프트 드롭, 하드 드롭 거리, 복합 점수	입력 직후 패널 값이 새로고침 없이 갱신된다.
하드 드롭	실제 이동 셀×2, 이동 거리 0이면 0점, 고정 처리는 한 번	`Space` 직후 충돌 직전 위치에 고정된다.
이름	trim, 보이는 문자 2~10자, 허용 문자, 잘못된 값 저장 거부	오류가 입력란 옆에 보이고 한 게임은 한 번만 저장된다.
정렬·TOP 10	점수↘, `playedAt` ↗, 안정 정렬, 11개 입력 > 10개	동점 순서와 각 행의 순위·이름·점수·줄·날짜가 맞다.
저장·복구	저장 후 재로드, 빈·손상·비배열 JSON > 빈 목록	새로고침 유지, 초기화 취소·확인, 손상 데이터에서 정상 시작

자동 검사는 점수·이름·정렬을 **순수 함수**, 저장을 **localStorage 어댑터**로 분리해 고정 입력과 기대 결과를 비교합니다.

회귀: SPEC 00~02의 화면·이동·충돌·줄 제거·게임오버·재시작과 `test.html` 검사가 모두 통과해야 한다.

전체 회귀 검증과 회고

- SPEC 00의 화면·초기 상태와 `test.html` 검사
- SPEC 01의 이동·낙하·회전
- SPEC 02의 충돌·줄 제거·게임오버·재시작
- SPEC 03의 점수·하드 드롭·리더보드

마지막으로 `MEMORY.md`를 함께 읽으며 답합니다.

1. 코드가 아니라 스펙을 고쳐 해결한 문제는 무엇인가?
2. 자동 검사가 잡은 문제와 사람이 잡은 문제는 무엇인가?
3. 같은 루프를 다시 돌릴 때 제거할 애매함은 무엇인가?

심화: SPEC 04를 자동 루프로 돌리기

레벨과 속도

- 시작 레벨 `1`
- `floor(제거 줄/10)+1`
- 낙하 간격 `max(100, 700−(레벨−1)×60)` ms
- 줄 점수만 현재 레벨 배수
- 드롭 점수에는 배수 없음

다음 블록

- 현재 블록 생성 시 다음 블록 하나 확정
- 모양과 색을 패널에 표시
- 고정 후 미리 본 블록을 실제 생성
- 새 다음 블록을 뽑아 미리보기 갱신

일시정지

- `PLAYING ↔ PAUSED` : `P`
- 정지 중 낙하·이동·회전·드롭 무시
- 다른 상태의 `P` 는 무시
- 재개 후 활성 타이머는 하나

자동 검사와 완료 조건이 준비되었다면 Ralph 루프를 최대 세 번까지 사용해봅니다.

자동 반복은 좋은 스펙을 대신하지 않습니다. 좋은 스펙이 준비됐을 때 반복 비용을 줄여줍니다.

SPEC 04 — 완료 조건과 검증

자동 검증

- 제거 줄 `0·9·10·19·20` 의 레벨을 확인한다.
- 높은 레벨에서도 낙하 간격은 100ms 아래로 내려가지 않는다.
- 줄 점수만 레벨 배수, 소프트·하드 드롭 점수는 그대로다.
- 고정 난수로 `미리보기 → 실제 생성 → 새 미리보기` 순서를 확인한다.
- 가짜 타이머로 정지 중 무낙하와 재개 후 타이머 하나를 확인한다.
- 모든 상태에서 `P` 입력의 허용·거부 결과를 확인한다.

직접 검증

- 9줄에서 경계를 넘어 레벨·속도·점수 표시 변화를 확인한다.
- 여러 블록을 고정하며 미리보기와 실제 블록을 비교한다.
- 정지 중 자동 낙하와 모든 조작이 멈추는지 확인한다.
- `P` 를 반복 입력한 뒤에도 낙하 속도가 중복되지 않는다.
- `READY`, `LANDED`, `GAME_OVER` 에서는 `P` 가 상태를 바꾸지 않는다.

회귀: SPEC 00~03 전체와 `test.html` 의 모든 검사가 통과해야 완료 신호를 허용한다.

먼저 끝났다면: 두 갈래 중 하나

SPEC 00~04 회귀 검증을 모두 통과한 뒤, 성격이 다른 두 과제 중 하나를 고릅니다

A. 테트리스 확장

유령 블록·블록 홀드·7-bag 중 **한 가지**를 골라 새 SPEC을 직접 설계하고 돌립니다.

- `SPEC_05_[이름].md` 에 목표·범위 밖·예외·완료 조건·검증
- 자동 검사 3개 이상, 직접 검증 2개 이상을 먼저
- 최대 세 번 실행, 같은 실패 두 번이면 중단

루프를 한 바퀴 더, 더 어려운 기능으로.

B. 내 업무 루프

오늘 만든 하네스에 **내 업무 하나**를 태워 봅니다.

- 업무 1건을 SPEC으로 분해
- 완료 조건 5개 이상, 자동 1개·직접 2개 이상 검증 설계
- `/spec-run` 으로 **최소 한 바퀴** 실행, `MEMORY.md` 기록

루프의 모양은 같고, 재료만 게임에서 업무로.

B는 잘하는 사람의 보상이 아니라 **다른 성격의 과제**입니다. 다음 장의 판정을 통과하는 업무가 있을 때만 갑니다. 없으면 A가 정답입니다.

B. 이 업무는 루프가 될 수 있는가

5분 안에 스스로 판정합니다. 하나라도 ✗면 오늘은 A트랙입니다

#	질문	통과 기준
1	"제대로 됐다"를 **사람이 아닌 것**이 판정할 수 있는가	파일 존재, 형식 일치, 테스트 통과, 숫자 비교 중 하나
2	입력이 매번 **같은 형태**로 들어오는가	커밋 로그, 로그 파일, 스펙 문서처럼 구조가 고정
3	한 번 실행이 **작은 단위**로 끝나는 크기로 자를 수 있는가	함수 하나, 규칙 하나, 문서 한 절
4	결과가 틀렸을 때 **되돌릴 수 있는가**	운영 반영·외부 발송·결제가 **아님**
5	실습 자료를 **사내 데이터 없이** 만들거나 가릴 수 있는가	합성 샘플, 마스킹, 공개 저장소

B트랙 산출물 — 업무 SPEC 00 · 검증 설계 · `MEMORY.md` 1회전 기록 · 자동화 준비도 자기 판정(현재 단계와 다음 단계 하나)

업무 SPEC 00 템플릿 열기

B. 고를 수 있는 시드 과제

"내 업무를 가져오세요"만으로는 절반이 못 고릅니다. 여섯 개 다 완료 조건이 기계로 검증됩니다

시드 과제	완료 조건 예시	자동 검증
릴리즈 노트 생성 루프	커밋 로그 범위 > 분류된 초안, 항목마다 커밋 해시 링크	형식 스키마 검사, 누락 커밋 0건
테스트 보강 루프	미커버 함수 1개씩 테스트 추가 > 통과	커버리지 수치 증가
안티패턴 정리 루프	규칙 1종씩 위반 지점 수정 > 작은 diff	린터 위반 수 감소, 기존 테스트 유지
로그·크래시 분류 루프	샘플 로그 > 분류 규칙 적용 > 오분류 피드백 반영	라벨링된 정답셋 대비 일치율
문서·코드 동기화 루프	API 스펙과 구현 차이 검출 > 문서 갱신	스펙 파서 diff 0건
코드리뷰 체크리스트 루프	룰 파일 기준 자동 점검 > 리뷰 코멘트 초안	룰별 판정 결과표 생성

실제 데이터 대신 **합성·마스킹 샘플**로 시작합니다. 사내 데이터를 강의장 노트북에 옮기지 않습니다.

A. SPEC 05+ — 선택 가능한 게임 기능

유령 블록

- 현재 위치·회전에서 충돌 없는 최하단에 반투명 표시
- 생성·이동·회전할 때마다 위치 갱신
- 충돌·고정·줄 제거·점수 계산에는 포함하지 않음
- `PAUSED`, `GAME_OVER` 에서 표시하지 않음

완료 조건

하드 드롭의 최종 좌표가 유령 블록 좌표와 같다.

블록 홀드

- `PLAYING` 에서 `C` 로 현재 블록 보관·교환
- 빈 보관함이면 미리 본 다음 블록을 생성
- 기존 보관 블록과 교환하면 미리보기는 유지
- 한 블록이 고정되기 전에는 한 번만 허용
- 교환 블록은 초기 위치·회전으로 생성
- 재시작하면 보관함을 비움

완료 조건

고정 전 두 번째 `C` 는 상태를 바꾸지 않는다.

7-bag 랜덤

- I·J·L·O·S·T·Z를 묶음마다 한 번씩 포함
- 주입 가능한 난수로 묶음을 섞음
- 일곱 개를 소진하면 새 묶음 생성
- 미리보기와 실제 생성이 같은 큐를 사용

완료 조건

14개를 뽑으면 각 7개 묶음에 일곱 종류가 정확히 한 번씩 있다.

세 기능을 함께 구현하지 않습니다. 선택한 한 기능의 상태·입력·예외만 새 SPEC에 기록합니다.

A. SPEC 05+ — 완료 조건과 검증

기능	자동 검증	직접 검증·예외
유령	`calculateGhostY`를 빈 보드·쌓인 블록·벽 회전 fixture로 확인하고 하드 드롭 좌표와 비교	이동·회전할 때 표시가 갱신되고, 정지·게임오버에서는 보이지 않는다.
홀드	빈/채운 보관함 전이, 두 번째 입력 거부, 고정 후 재허용, 재시작 초기화를 확인	첫 홀드 > 재입력 > 고정 > 교환을 재현하고, 교환 위치가 막히면 게임오버다.
7-bag	고정 난수로 14개를 뽑아 각 7개 집합과 재현 가능한 순서를 확인	묶음 경계에서도 미리보기와 실제 생성 순서가 항상 일치한다.

새 SPEC의 필수 항목

- 사용자 가치·범위 밖 / 화면·입력·상태 변화·우선 규칙
- 경계·재시작·일시정지·게임오버 예외
- `시작 상태 + 입력 → 기대 결과` 완료 조건 5개 이상
- 자동 검사 3개 이상, 직접 검증 2개 이상

회귀: SPEC 00~04의 완료 조건이 하나라도 깨지면 SPEC 05+는 미완료입니다.

06

게임에서 업무로 루프 옮겨가기

결과를 공유하고, 내 업무의 SPEC 00 작성하기

결과 공유

테트리스 기능 소개보다 **루프가 어떻게 발전했는지** 공유합니다.

1. 가장 많이 고친 SPEC과 그 이유
2. 자동 검사는 통과했지만 사람이 발견한 문제
3. `MEMORY.md` 가 다음 반복에 도움을 준 순간
4. 루프를 멈추고 사람의 판단이 필요했던 순간
5. A트랙의 게임 확장 또는 B트랙의 업무 루프에서 내린 판단

피드백 질문: 이 결과를 다시 만들 수 있는 근거가 남아 있나요?

테트리스의 구조를 업무로 번역합니다

테트리스 루프	업무 루프 예시
SPEC 00: 화면과 골격	보고서 목차, 표 컬럼, 파일명과 저장 위치
SPEC 01: 기본 입력 처리	정상 입력을 분류하고 초안을 만드는 규칙
SPEC 02: 충돌과 실패	누락, 중복, 잘못된 형식, 판단 불가 처리
SPEC 03: 점수와 기록	처리 상태, 품질 지표, 변경 이력 저장
SPEC 04: 레벨과 개선	반복 결과에 따른 우선순위·규칙·주기 개선
선택 실습 B: 내 업무	오늘 만든 하네스에 업무 SPEC을 태워 1회전

업무도 처음부터 완전 자동화하지 않습니다. **가장 작은 결과물부터 검증하며 기능을 붙입니다.**

자동화 준비도 정하기

단계	상태	의미
0	즉석 프롬프트	매번 사람이 새로 설명
1	반복 가능한 SPEC	입력·출력·완료 조건 고정
2	사람이 시작하는 루프	한 번씩 실행하고 상태 파일에 기록
3	제한된 자동 반복	자동 검사와 최대 반복 횟수 존재
4	부분 자동 처리	낮은 위험 결과만 자동 반영
5	전체 자동화 검토	검증·권한·복구 체계가 성숙한 뒤 검토

오늘 테트리스 필수 실습의 목표는 **2단계**, SPEC 04 심화 실습은 **3단계**입니다. B트랙은 자기 업무가 지금 **어느 단계인지 판정하고, 다음 단계 하나**를 정하는 것까지가 산출물입니다.

업무 루프는 위험도가 다르므로 오늘 바로 3단계로 올리지 않습니다.

좋은 첫 업무 루프는

사람이 승인할 수 있는
작은 결과와 검증 근거를 안정적으로 남깁니다

- 테트리스는 기능이 늘어났지만, 루프의 모양은 변하지 않았습니다.
- 하네스는 반복되는 부탁을 규칙으로 바꿨습니다.
- SPEC은 기대를 고정했고, 검증은 완료의 근거를 만들었습니다.
- `MEMORY.md` 는 실패를 다음 반복의 입력으로 바꿨습니다.
- 자동 반복은 그 구조가 안정된 뒤에야 추가했습니다.

내일부터는 여러분의 업무에 작은 SPEC 한 장부터 굴려보세요!

B트랙에서 쓴 (또는 내일 채워볼) 자료: 업무 SPEC 00 템플릿